Lean 4 for People Who Prove Things Elsewhere

A primer and cookbook for the mathematically fluent newcomer

Retrieved and assembled 2026-08-10

Abstract

This primer introduces the Lean 4 theorem prover and programming language to a reader with a strong mathematics and software-engineering background who has never used an *interactive theorem prover* (ITP)—a tool in which a human writes a proof step by step while the machine checks every move. We situate Lean against two systems such a reader may know, TLA$^+$ and Haskell; show through short, real code fragments what its two halves (a programming language and a proof assistant) actually look like; answer the practical questions of whether Lean is a general-purpose language and how it interoperates with C; and close with a worked, fully machine-checked theorem. Every Lean fragment in this document type-checks under Lean 4.33 with no external library, and lives in the companion `primer/` project.

Contents

1 What Lean 4 is — **2**
- 1.1 The functional-programming half, in code — 2
- 1.2 What "theorem prover" means, concretely — 2

2 Situating Lean: TLA$^+$ and Haskell — **4**

3 Is Lean a general-purpose language? IO, data, C interop — **4**

4 Tooling and editors — **5**

5 The proof experience and the crypto-relevant layer — **6**

6 Verifying properties of your own code — **6**

7 A worked theorem: the Josephus closed form — **7**

8 Illuminating examples: discrete probability — **8**

9 mathlib, and where to go next — **9**

Glossary — **9**

A Syntax and tactics reference — **10**

References — **11**

1 What Lean 4 is

Lean 4 is two things in one language: a **dependently-typed functional programming language** and an **interactive theorem prover**. The same syntax, type system, and tooling serve both jobs; you do not switch dialects to go from writing a program to writing a proof.

1.1 The functional-programming half, in code

"Functional programming language" means the ordinary things—definitions, pattern matching, recursion, algebraic data types—look like this:

```
def double (n : Nat) : Nat := n + n           -- a function Nat → Nat

def fib : Nat → Nat                            -- recursion with pattern matching
  | 0 => 0
  | 1 => 1
  | n + 2 => fib n + fib (n + 1)

inductive Tree (α : Type) where               -- an algebraic data type
  | leaf : Tree α
  | node : Tree α → α → Tree α → Tree α

#eval fib 10                                  -- 55  (#eval runs the code)
```

"Dependently-typed" is the part without a Haskell or ML analogue: a *type may mention a value*. The length of a vector can live *in its type*, so the type-checker—not a runtime check—enforces that appending an m-vector and an n-vector yields an $(m + n)$-vector:

```
-- `Vector α n` is a list of α whose length n is part of the type.
example : Vector Nat 3 := #v[10, 20, 30]         -- ok
-- example : Vector Nat 3 := #v[10, 20]          -- TYPE ERROR: length 2 ≠ 3

-- The type of `append` promises the output length is the sum of the inputs':
#check (Vector.append : Vector α m → Vector α n → Vector α (m + n))
```

Because a type can depend on values, a type can also *encode a mathematical statement*—and that is precisely the bridge to theorem proving (Section 1.2). A second consequence, invisible above but load-bearing: Lean functions are **total by default**. The system checks that each recursion terminates (`fib` above is accepted because its arguments decrease); a function that might loop forever or crash is rejected unless you explicitly opt out. Totality is what makes it sound to read a function as a proof.

1.2 What "theorem prover" means, concretely

The link between the two halves is the **Curry–Howard correspondence**: a *proposition* is a *type*, and a *proof* of it is a *term* (a value) of that type. Proving P therefore means *constructing a term whose type is* P, and checking a proof is the same operation a compiler already performs—verifying that a term has the type it claims.

An aside on `rfl`. You will see `rfl` everywhere, so it is worth pinning down. It is the *one* canonical proof that a thing equals itself: its type says `a = a` holds for every `a`. It proves a goal `x = y` exactly when `x` and `y` *compute to the same value*—then, after evaluation, both sides are literally the same thing, and "$a = a$" applies. So `2 + 2 = 4` is closed by `rfl` because `2 + 2` reduces to 4; but `a + b = b + a` is *not*, because the two sides do not reduce to a common form (there you reach for `omega`).

A real proof, end to end. Something less trivial than `2 + 2`: the sum of the first n odd numbers is n^2. The proof is a genuine induction with a real algebraic step.

```
def sumOdd : Nat → Nat
  | 0 => 0
  | n + 1 => sumOdd n + (2 * n + 1)      -- add the (n+1)-th odd number, 2n+1

theorem sumOdd_eq (n : Nat) : sumOdd n = n * n := by
  induction n with
  | zero => rfl                          -- base: sumOdd 0 = 0 = 0*0, by computation
  | succ k ih =>
      have h : (k + 1) * (k + 1) = k * k + 2 * k + 1 := by
        simp only [Nat.succ_mul, Nat.mul_succ]; omega   -- the one nonlinear fact
      simp only [sumOdd, ih, h]          -- k*k + (2*k+1) = k*k + 2*k + 1
      omega
```

The two cases are the two ways to build a `Nat`: `zero` (closed by `rfl`, since both sides compute to 0) and `succ k`, where the *induction hypothesis* `ih : sumOdd k = k * k` is available and we extend it by one step.

Reading a tactic proof as an ordinary one

None of those tokens are "assume" or "therefore"—tactic mode is *imperative* (commands to a proof engine), and it names its routine steps after the automation that discharges them. But each tactic *is* a step of the textbook proof:

`induction n`	"by induction on n"; the two cases `zero` and `succ k`, with `ih` the hypothesis
`rfl` *(zero)*	base case: both sides compute to 0
`simp only [sumOdd, ih, h]`	unfold the *definition*, use the *hypothesis* `ih`, apply the *algebra* `h`
`omega`	finish the leftover arithmetic

Written as an explicit chain (Lean's `calc`), the *same* proof reads almost verbatim like the textbook argument $s(k{+}1) = s(k) + (2k{+}1) = k^2 + (2k{+}1) = (k{+}1)^2$:

```
calc sumOdd (k + 1)
    = sumOdd k + (2*k+1)  := rfl                -- definition
  _ = k*k + (2*k+1)       := by rw [ih]          -- induction hypothesis
  _ = (k+1)*(k+1)         := by rw [h]; omega    -- algebra
```

And "by induction" is not a trusted keyword: it builds the term `Nat.recAux base step n`, an application of `Nat`'s induction principle—itself proved once, in the kernel, from how `Nat` is defined.

What the system generates. A `by` block is a script; the proof is the *term* it produces, which is what the kernel checks. Take the two above.

You type:

```
theorem two_plus_two : 2 + 2 = 4 := by rfl
```

Lean generates (`#print two_plus_two`):

```
Eq.refl (2 + 2)
```

Why it proves the theorem: `Eq.refl` is the one constructor of equality, of type `a = a`. `Eq.refl (2 + 2)` therefore has type `2 + 2 = 2 + 2`—and since `2 + 2` and `4` reduce to the same value, that type *is* `2 + 2 = 4`. The kernel confirms the typing.

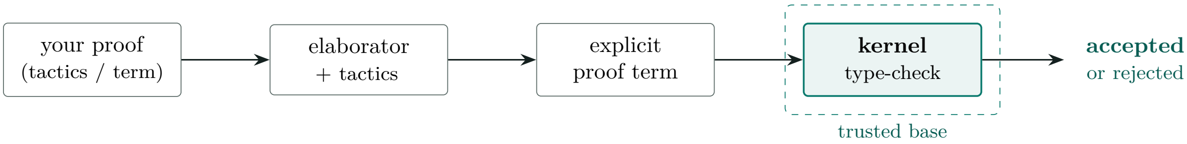

Figure 1: How a proof is checked. Everything left of the dashed box may be elaborate or buggy; soundness rests only on the kernel re-checking the final term. Boxes are artifacts; solid arrows are transformations.

You type: the `sumOdd_eq` induction above. *Lean generates* (`#print sumOdd_eq`, plumbing elided):

```
fun n => Nat.recAux (Eq.refl (sumOdd 0))    -- the `zero` case
           (fun k ih => ...)                  -- the `succ` case, using ih
           n
```

Why it proves the theorem: `Nat.recAux` is induction on `Nat` as a function—give it a proof for `0` and a map from a proof for `k` to one for `k+1`, and it returns a proof for every `n`. Its type here is exactly `sumOdd n = n * n` for all `n`, which the kernel checks.

The kernel—a small, fixed core (a few thousand lines)—is the only thing trusted; tactics live outside it, so a buggy tactic cannot force a false theorem past it (Figure 1) [6]. A proof may also contain `sorry`, a hole that admits any goal; Lean warns while any remains.

2 Situating Lean: TLA$^+$ and Haskell

The fastest way to understand Lean is by contrast with tools answering *different* questions (Table 1).

Versus TLA$^+$. TLA$^+$ is a *specification language*: you model a system—often concurrent or distributed—as a state machine and state properties in temporal logic, then check them primarily by **model checking**, in which the TLC checker exhaustively explores a *finite* state space and prints a counterexample trace when a property fails. Lean does not explore states; it **proves** statements deductively over possibly-infinite objects. The mental shift is bounded-and-automatic-with-counterexamples (TLC) versus unbounded-and-deductive-without-counterexamples (Lean). Lean's `#eval` and `decide` can feel faintly model-checker-like on finite decidable propositions, but that is evaluating a decision procedure, not exploring a temporal state space. Reach for TLA$^+$ to ask "does my design have a bad interleaving?"; reach for Lean to ask "is this true for all inputs, provably?".

Versus Haskell. Much will feel familiar: both are pure and functional with type classes, algebraic data types, `do` notation, monads, and strong inference. The differences are the point. Lean has full *dependent* types (Section 1.1); it is total by default where Haskell is lazy and partial (`undefined` inhabits every Haskell type); and its types can state theorems, which Haskell's cannot in any practical sense. Treat Haskell as good intuition for the syntax and the functional style, then add "types can talk about values, and functions must terminate."

3 Is Lean a general-purpose language? IO, data, C interop

Yes. Lean 4 is a genuine general-purpose language, not a proof-only toy. It compiles to C and then to native code, has an `IO` monad for real input/output, ships arrays, hash maps, strings and

	TLA$^+$	Lean 4	Haskell
Primary job	specify systems & temporal behavior	prove theorems; write verified programs	write programs
How you check	model-check a *finite* state space (TLC); finds counterexamples	*deduce* over infinite objects; no counterexamples unless you search	type-check; run tests
Types	untyped set theory	full *dependent* types; can express theorems	Hindley–Milner + extensions
Totality	—	total by default (termination-checked)	lazy & partial
Ask it	"bad interleaving?"	"true for all inputs, provably?"	"elegant program?"

Table 1: Three overlapping but distinct questions. Lean is the deductive, unbounded one.

`ByteArray` for ordinary data manipulation, and is substantial enough that *the Lean compiler and much of its standard library are written in Lean itself* [6]. A minimal program:

```
def main : IO Unit := do
  let stdin ← IO.getStdin
  let line ← stdin.getLine
  IO.println s!"you said: {line.trim}"         -- string interpolation
```

Calling C from Lean. The `@[extern "sym"]` attribute binds a Lean declaration to an external C symbol; you give a Lean signature and type-level model, and link the C implementation:

```
@[extern "lean_sqrt"]         -- calls C function `double lean_sqrt(double)`
opaque cSqrt (x : Float) : Float
```

Calling Lean from C. The `@[export sym]` attribute exposes a Lean function under an unmangled, C-callable name; you compile the Lean code and link against the Lean runtime (`libleanshared`), which provides garbage collection and the reference-counting discipline Lean's compiled objects use:

```
@[export my_add]              -- becomes C-callable `lean_object* my_add(...)`
def myAdd (a b : Nat) : Nat := a + b
```

So interoperation runs both directions. The one caveat is the runtime: Lean-compiled code is C *plus* the Lean runtime and its object model (reference-counted `lean_object*`, with each parameter either owned or borrowed), so embedding Lean in a C program links a runtime rather than dropping in a freestanding `.o` [7]. Lean is thus usable for library building and data-processing tools; what it is *not* is a drop-in systems language with no runtime.

4 Tooling and editors

Lean 4 ships a **language server** (the `lean --server` command, speaking the *Language Server Protocol*, LSP), so editor support is a matter of an LSP client plus a plugin that renders the *infoview*—the live display of the proof goal at the cursor.

- **VS Code** has the most complete, first-party extension. (Noted for completeness; a preference against it does not block anything below.)
- **Emacs** (`lean4-mode`) and **Neovim** (`lean.nvim`) are actively maintained LSP clients with infoview support—the mainstream non-VS-Code choices.

- **Xcode: there is no Lean plugin.** Xcode does not expose a general third-party-language / LSP extension point, and no Lean 4 Xcode integration exists as of 2026. If you want to stay in the Apple toolchain, the practical route is the command line (below) plus an LSP-capable editor (Emacs/Neovim) alongside it.
- **Command line only** is entirely viable, and is how the companion project is checked:

```
lake build                          -- type-check the whole project = check all
    proofs
lake env lean Primer/Basics.lean   -- run one file, printing its #eval / #check
    output
```

There is no separate "test" step for the proofs: a clean `lake build` *is* the verification, because the build is exactly the kernel checking every term (Section 1.2). One practical caution learned while assembling this primer: `lake build` can report success from a cached build artifact; when you want the ground truth for a single file, run `lake env lean <file>`, which re-elaborates from source.

5 The proof experience and the crypto-relevant layer

Proofs are written either in *term mode* (hand Lean the term directly) or, more often, in *tactic mode*: a `by` block runs **tactics**—small commands that transform the goal while Lean assembles the underlying term. In an editor the infoview shows the evolving goal; on the command line you reason it out. A working vocabulary: `rfl` (true by computation), `decide` (run a decision procedure on a finite/decidable goal), `simp` (rewrite with a lemma database), `omega` (linear arithmetic over integers and naturals), and `induction`. *Every* construct, tactic, and lemma used in this primer's code is glossed in the reference (Section A); reach for it whenever a listing shows something unfamiliar.

Lean's `BitVec n` type—fixed-width machine words with wraparound arithmetic—is the layer where cryptography lives, and it is directly relevant to the companion survey of SHA-256 formalization. SHA-256's whole primitive layer is a few total functions:

```
def rotr (x : BitVec 32) (n : Nat) : BitVec 32 := (x >>> n) ||| (x <<< (32 - n))
def Ch     (x y z : BitVec 32) := (x &&& y) ^^^ ((~~~x) &&& z)
def Maj    (x y z : BitVec 32) := (x &&& y) ^^^ (x &&& z) ^^^ (y &&& z)
def Sigma0 (x : BitVec 32) := rotr x 2 ^^^ rotr x 13 ^^^ rotr x 22

-- Over ALL 256 eight-bit words, brute force (`decide`) settles this identity:
example : ∀ x : BitVec 8, x ^^^ x = 0#8 := by decide
-- Over 32 bits it is 2^32 cases; there `decide` is hopeless and the tactic is
-- `bv_decide` - bit-blast to SAT and check an LRAT certificate in-kernel.
```

That last tactic, `bv_decide`, is verified bit-blasting: it hands the goal to a SAT solver and then re-checks the solver's refutation certificate with a Lean-verified checker, so the untrusted solver never enters the trusted base [3]. It is why Lean is now a strong platform for bit-level cryptographic reasoning even though a full software SHA-256 correctness proof in Lean does not yet exist—the subject of the companion survey.

6 Verifying properties of your own code

Set expectations precisely. Lean is excellent for proving properties of *pure functions*, data-structure *invariants*, and math-heavy *kernels* that you model or re-implement in Lean; there you can prove a statement for *all* inputs. Lean is *not* a push-button verifier for an arbitrary existing Rust, Python, or C repository—that is the domain of SMT-backed tools (*Satisfiability Modulo Theories*: automatic decision procedures over arithmetic, arrays, and the like) such as Dafny or Why3, or of model checkers such as TLA$^+$. The productive loop is: model the critical pure core in Lean, prove it there, and treat that as your trustworthy reference. A representative shape—a fast implementation proved equal to an obvious reference model:

```
def sumTo : Nat → Nat                              -- reference model
  | 0 => 0
  | n + 1 => (n + 1) + sumTo n

def sumToAcc : Nat → Nat → Nat                      -- optimized tail-recursive version
  | 0,      acc => acc
  | n + 1, acc => sumToAcc n (acc + (n + 1))

theorem sumTo_correct (n : Nat) : sumToAcc n 0 = sumTo n := by
  simp [sumToAcc_eq]                               -- via a generalized accumulator lemma
```

7 A worked theorem: the Josephus closed form

To show a non-trivial result carried all the way to a machine-checked proof, we formalize a classic from Knuth's *Concrete Mathematics* [5]: the Josephus problem. Of n people in a circle every second is eliminated; the survivor's position $J(n)$ satisfies

$$J(1) = 1, \qquad J(2n) = 2J(n) - 1, \qquad J(2n+1) = 2J(n) + 1,$$

with the memorable closed form: writing $n = 2^m + \ell$ with $0 \le \ell < 2^m$,

$$\boxed{J(2^m + \ell) = 2\ell + 1}$$

equivalently "$J(n)$ is the binary numeral of n rotated left by one bit" (e.g. $13 = 1101_2 \mapsto 1011_2 = 11 = J(13)$). The bit-rotation reading is a direct callback to `rotr` in Section 5, which is why this problem is pleasantly close to the SHA-256 world. Here is the recurrence and the general theorem, both machine-checked in pure Lean (no library). The listing below is **what you type**—the definition and a tactic proof; any construct in it is glossed in Section A:

```
def josephus : Nat → Nat
  | 0 => 0
  | 1 => 1
  | n + 2 =>
      have : (n + 2) / 2 < n + 2 := Nat.div_lt_self (by omega) (by omega)
      if (n + 2) % 2 = 0 then 2 * josephus ((n + 2) / 2) - 1
                         else 2 * josephus ((n + 2) / 2) + 1

theorem josephus_pow (m : Nat) :
    ∀ l, l < 2 ^ m → josephus (2 ^ m + l) = 2 * l + 1 := by
  induction m with
  | zero => intro l hl; rw [Nat.pow_zero] at hl ⊢; have : l = 0 := by omega
            subst this; simp [josephus]
  | succ m ih =>
      intro l hl
      have hpow : (2 : Nat) ^ (m + 1) = 2 * 2 ^ m := by rw [Nat.pow_succ]; omega
      have hpos : 0 < (2 : Nat) ^ m := Nat.two_pow_pos m
      rcases (show l % 2 = 0 ∨ l % 2 = 1 by omega) with hpar | hpar
      · have hjm : l / 2 < 2 ^ m := by omega
        have hk : (2:Nat) ^ (m+1) + l = 2 * (2 ^ m + l / 2) := by omega
        rw [hk, josephus_even (2 ^ m + l / 2) (by omega), ih (l / 2) hjm]; omega
```

```
      · have hjm : 1 / 2 < 2 ^ m := by omega
        have hk : (2:Nat) ^ (m+1) + 1 = 2 * (2 ^ m + 1 / 2) + 1 := by omega
        rw [hk, josephus_odd (2 ^ m + 1 / 2) (by omega), ih (1 / 2) hjm]; omega
```

Everything after `:= by` is a script of *tactics*—your instructions for building the proof, not the proof itself. **What Lean generates** from it is an ordinary term, which is what the kernel actually checks. Printing it (`#print josephus_pow`, trimmed here) shows the `induction` tactic became an application of `Nat`'s recursor, with your two cases as its arguments:

```
-- WHAT LEAN GENERATES (the proof term the kernel checks) - abbreviated:
theorem josephus_pow : ∀ (m l : Nat), l < 2 ^ m → josephus (2 ^ m + l) = 2*l + 1 :=
fun m =>
  Nat.recAux
    (fun l hl => ...)          -- the m = 0 case you wrote as `| zero => ...`
    (fun m ih l hl => ...)     -- the m+1 case you wrote as `| succ m ih => ...`
    m
-- (the real term fills each `...` with ~15 lines of Eq.mpr/congrArg/... plumbing)
```

You never write that term by hand—that is the whole point of tactic mode. But it exists, and the kernel re-checks it; the tactics are only a convenient way to produce it. The proof itself is a textbook induction on m that splits on the parity of ℓ and steps down through the recurrence—exactly the *Concrete Mathematics* argument, now checked by the kernel rather than by a careful reader. The full file also machine-checks (over $1 \le n \le 2000$, via `native_decide`) that the recurrence, the closed form, and the one-bit-rotation form agree. This is the kind of concrete, self-contained result that Lean makes routine—and, notably, it required no mathematics library at all.

8 Illuminating examples: discrete probability

Probability makes a nice showcase because the surprising facts become things the machine can *check*. Over a finite, equally-likely sample space a probability is just `favorable / total` (two naturals), and a claim "$p > 1/2$" is the exact inequality `2 * favorable > total`—no real numbers required.

Monty Hall. Enumerate the nine equally-likely `(car, firstPick)` games. After the host opens a goat door, switching wins *exactly when your first pick was wrong*, so it wins $6/9 = 2/3$ against staying's $3/9$:

```
def games : List (Nat × Nat) :=              -- all 9 (car, pick) pairs
  [0,1,2].flatMap (fun car => [0,1,2].map (fun pick => (car, pick)))

def countWhere (xs : List α) (p : α → Bool) : Nat := (xs.filter p).length

example : games.length = 9
        ∧ countWhere games (fun g => g.1 == g.2) = 3        -- staying wins
        ∧ countWhere games (fun g => g.1 != g.2) = 6 := by  -- switching wins
  decide
```

The birthday paradox. Among n people (365 equally-likely birthdays), all differ with probability $(\prod_{i<n}(365 - i))/365^n$; a shared birthday becomes more likely than not once `2 * numerator < 365^n`. The numbers are ~60 digits, so `native_decide`—which *compiles* the check to native code rather than reducing it in the kernel—does the arithmetic (Lean's `Nat` is arbitrary-precision). It confirms the threshold is *exactly* 23 people:

```
def noCollisionNum : Nat → Nat
  | 0 => 1
  | n + 1 => (365 - n) * noCollisionNum n

theorem birthday_threshold :
    2 * noCollisionNum 23 < 365 ^ 23 ∧ 2 * noCollisionNum 22 ≥ 365 ^ 22 := by
  native_decide
```

The companion `Primer/Probability.lean` adds De Méré's 1654 dice problem—the wager that launched the subject—machine-checking that a bet paying off on "one 6 in four rolls" clears 1/2 while "one double-6 in twenty-four rolls" falls just short. Every construct these listings use is glossed in Section A.

9 mathlib, and where to go next

mathlib is the community's large, coherent library of formalized mathematics—algebra through analysis and beyond—with the lemmas and automation to match [1]. It is what you pull in when a proof rests on real mathematics; it is also a heavy dependency (its prebuilt object cache is several gigabytes, and the first `import Mathlib` takes minutes). Everything in this primer *deliberately* uses none of it—core Lean plus its standard library, *Batteries*, is enough for algorithmic and bit-level work, and the project stays instant and offline. That is a choice, not a limitation: when a proof genuinely needs mathematics, you add `require mathlib` to your `lakefile`, run `lake exe cache get` to fetch the prebuilt library, and its full power is available.

What it buys you. The companion `mathlib-tour/` project makes the trade-off concrete. The one awkward step in Section 1.2—proving $(k+1)^2 = k^2 + 2k + 1$ by hand with `simp only [Nat.succ_mul, Nat.mul_succ]; omega`—collapses to a single `ring`. It also states things core Lean cannot: an actual finite sum $\sum_{i<n}(2i+1) = n^2$, the irrationality of $\sqrt{2}$, and dice probability as a genuine rational over a `Finset` rather than the integer counting the primer used to stay library-free.

Where to go next. *Theorem Proving in Lean 4* [2] for the proof language, *Functional Programming in Lean* [4] for the programming side, and *Mathematics in Lean* [1] for working with mathlib. The companion `primer/` project holds every core-Lean example above as runnable, type-checked source, and `mathlib-tour/` holds the mathlib ones.

Glossary

Kernel
the small trusted core that checks a term has its claimed type; the sole component that must be trusted for soundness.

Term / Proposition
a term is a value/program; a proposition is a statement realized as a type whose terms are its proofs (Curry–Howard).

Tactic
a command that transforms the proof goal, building a proof term (e.g. `simp`, `omega`, `induction`).

Dependent type
a type that mentions a value (e.g. length-indexed vectors); what lets types state theorems.

ITP / ATP
interactive theorem prover (human-guided, machine-checked) versus automated theorem prover (search-driven).

LSP / FFI
Language Server Protocol (editor integration); Foreign Function Interface (calling between languages, here Lean↔C).

`decide` / `bv_decide`
run a decision procedure on a decidable goal; the latter bit-blasts `BitVec` goals to a SAT solver and re-checks its certificate.

`elan` / `lake`
the toolchain manager (installs/selects Lean versions) and the build system / package manager.

A Syntax and tactics reference

Every language construct used in this primer's code listings, grouped and glossed. "Tactics" run inside a `by` block; "commands" (starting with `#`) run at compile time and print to the infoview.

Commands and declarations.

<table>
<tr><td><code>#eval e</code></td><td>run <code>e</code> and print the result</td></tr>
<tr><td><code>#check e</code></td><td>print the type of <code>e</code> without running it</td></tr>
<tr><td><code>#print n</code></td><td>print the definition or proof term of <code>n</code></td></tr>
<tr><td><code>def</code></td><td>define a value or function</td></tr>
<tr><td><code>theorem</code> / <code>example</code></td><td>state (and prove) a proposition; <code>example</code> is unnamed</td></tr>
<tr><td><code>inductive ... where</code></td><td>declare an algebraic data type by its constructors</td></tr>
<tr><td><code>opaque</code></td><td>declare a value with no body (e.g. one supplied externally)</td></tr>
<tr><td><code>@[extern "sym"]</code></td><td>implement this declaration with the C symbol <code>sym</code></td></tr>
<tr><td><code>@[export sym]</code></td><td>expose this declaration to C under the name <code>sym</code></td></tr>
</table>

Expression syntax.

<table>
<tr><td><code>fun x => e</code></td><td>anonymous function (lambda)</td></tr>
<tr><td><code>let x := e</code> / <code>let x <- e</code></td><td>local binding; the <code><-</code> form sequences a monadic (e.g. <code>IO</code>) step</td></tr>
<tr><td><code>do ...</code></td><td>sequence monadic actions</td></tr>
<tr><td><code>if c then a else b</code></td><td>conditional</td></tr>
<tr><td><code>| pat => e</code></td><td>a pattern-match arm (in a <code>def</code> or <code>match</code>)</td></tr>
<tr><td><code>(a, b)</code>, <code>p.1</code>, <code>p.2</code></td><td>a pair and its two projections</td></tr>
<tr><td><code>s!"... {x} ..."</code></td><td>string interpolation</td></tr>
<tr><td><code>0#8</code>, <code>0xff#32</code></td><td>a <code>BitVec</code> literal, written value<code>#</code>width</td></tr>
<tr><td><code>:</code> / <code>:=</code></td><td>type ascription / "is defined as"</td></tr>
</table>

Operators.

<table>
<tr><td><code>+ - * / %</code></td><td>arithmetic on <code>Nat</code> (<code>/</code> and <code>%</code> are truncating)</td></tr>
<tr><td><code>^</code></td><td>exponentiation</td></tr>
<tr><td><code>= < <= >=</code></td><td>(in)equality propositions; <code>==</code> and <code>!=</code> are Boolean</td></tr>
<tr><td><code>&&& ||| ^^^ ~~~</code></td><td>bitwise and / or / xor / not on <code>BitVec</code></td></tr>
<tr><td><code>>>> <<<</code></td><td>right / left shift on <code>BitVec</code></td></tr>
</table>

Tactics.

`rfl`	close a goal whose two sides compute to the same value
`decide`	decide a decidable goal by evaluating it
`native_decide`	like `decide`, but compiled to native code (large computations)
`simp` / `simp only [ls]`	simplify by rewriting (with *only* the lemmas `ls`)
`omega`	decide linear arithmetic over `Nat`/`Int`
`induction n with ...`	induct on `n`, one arm per constructor; an arm binds the induction hypothesis
`intro h`	move a `forall`/`->` premise into the context
`rw [eqs]`	rewrite the goal with equations; `rw [e] at h ⊢` also targets `h` and the goal
`subst h`	eliminate a variable using an equation hypothesis
`have h : P := pf`	introduce a proved sub-fact `h`
`show P`	restate the goal in a definitionally-equal form
`calc a = b := ...`	a chain of (in)equalities, each with its justification
`rcases e with h | h`	case-split on a disjunction or structure
`·`	focus the first remaining goal (one bullet per case)

Referenced library items. Their names describe them: `Nat.recAux` (the recursor / induction principle for `Nat`), `Eq.refl a` (the proof `a = a`), the arithmetic lemmas `Nat.succ_mul`, `Nat.mul_succ`, `Nat.pow_zero`, `Nat.pow_succ`, `Nat.two_pow_pos`, `Nat.div_lt_self`; the list operations `List.map`, `List.filter`, `List.flatMap`, `List.range`, `.length`; and `Vector.append` (length-tracking concatenation). The unfolding equation of a well-founded definition `f` is available as `f.eq_def`.

References

[1] Jeremy Avigad and Patrick Massot. *Mathematics in Lean*. Local copy: `../_refs/lean/mathematics_in_lean.pdf`, retrieved 2026-08-10. 2020. URL: https://leanprover-community.github.io/mathematics_in_lean/ (cit. on p. 9).

[2] Jeremy Avigad et al. *Theorem Proving in Lean 4*. HTML only: https://leanprover.github.io/theorem_proving_in_lean4/, retrieved 2026-08-10. 2021. URL: https://leanprover.github.io/theorem_proving_in_lean4/ (cit. on p. 9).

[3] Henrik Böving et al. "Interactive Bitvector Reasoning using Verified Bit-Blasting". In: *Proc. ACM Program. Lang.* 9.OOPSLA2 (2025). No free PDF located (ACM Digital Library, gated), retrieved 2026-08-10, pp. 3259–3285. DOI: 10.1145/3763167. URL: https://doi.org/10.1145/3763167 (cit. on p. 6).

[4] David Thrane Christiansen. *Functional Programming in Lean*. HTML only: https://leanprover.github.io/functional_programming_in_lean/, retrieved 2026-08-10. 2023. URL: https://leanprover.github.io/functional_programming_in_lean/ (cit. on p. 9).

[5] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics: A Foundation for Computer Science*. 2nd ed. Josephus problem, §1.3. Addison-Wesley, 1994 (cit. on p. 7).

[6] Leonardo de Moura and Sebastian Ullrich. “The Lean 4 Theorem Prover and Programming Language”. In: *Automated Deduction – CADE 28*. Vol. 12699. Lecture Notes in Computer Science. Local copy: `../_refs/lean/lean4-system-paper.pdf`, retrieved 2026-08-10. Springer, 2021, pp. 625–635. DOI: `10.1007/978-3-030-79876-5_37`. URL: `https://lean-lang.org/papers/lean4.pdf` (cit. on pp. 4, 5).

[7] The Lean Community. *The Lean Language Reference: Foreign Function Interface*. Online reference manual (HTML). HTML only, retrieved 2026-08-10. 2026. URL: `https://lean-lang.org/doc/reference/latest/Run-Time-Code/Foreign-Function-Interface/` (cit. on p. 5).